核心结论

`nextTick` 的本质：**把一个回调函数放到「本轮 DOM 更新之后、下一个事件循环的微任务/宏任务队列」中执行**，从而保证你在回调里拿到的是**已经更新完的 DOM**。

为什么需要 `nextTick`？

- Vue 的响应式更新是**异步批量更新**：
  - 多次修改数据，不会马上更新 DOM；
  - 而是先记录「哪些组件要更新」，放进一个队列，**在本轮事件循环结束前统一更新一次 DOM**。
- 因此：
  - 你在修改数据后**立刻访问 DOM**，拿到的还是旧的；
  - 用 `nextTick` 就是告诉 Vue：**“等你把该更新的 DOM 都更新完，再调用我的这个回调。”**

实现思想（通用逻辑）

用伪代码描述一下 Vue 内部的调度流程：

```
// 1. 数据变更触发：把需要更新的 watcher / effect 放进更新队列
queueWatcher(watcher) {
  if (!has[watcher.id]) {
    has[watcher.id] = true
    queue.push(watcher)
    // 保证本轮只安排一次 flush
    scheduleFlush()
  }
}

// 2. 安排一次异步的 flush 操作（本轮结束前执行）
scheduleFlush() {
  if (!pending) {
    pending = true
    // 关键：用微任务/宏任务把 flushQueue 放到“下一拍”
    nextTick(flushQueue)
  }
}

// 3. 真正执行：依次跑队列里的 watcher，做 DOM 更新
flushQueue() {
  pending = false
```

```
  const copies = queue.slice(0)
  queue.length = 0
  for (const watcher of copies) {
    watcher.run() // 里面会触发 render, 更新 DOM
  }
}

// 4. nextTick 本身也维护一个 callbacks 队列
let callbacks = []
let waiting = false

function nextTick(cb) {
  callbacks.push(cb)
  if (!waiting) {
    waiting = true
    timerFunc()   // 关键: 用 microtask / macrotask 安排 flushCallbacks
  }
}

function flushCallbacks() {
  waiting = false
  const copies = callbacks.slice(0)
  callbacks.length = 0
  for (const cb of copies) {
    cb()
  }
}

```

逻辑关系：

- 数据变化 → 放 watcher 队列 → `nextTick(flushQueue)` → DOM 更新完成
- 用户调用 `nextTick(userCb)` → 把回调放在 `flushCallbacks` 的队列中
- `flushQueue` 和 用户的 `userCb` **都在同一个** `flushCallbacks` **批次里**；
- 所以：**DOM 更新（由 watcher 引起）先执行，用户的** `nextTick` **回调后执行。**

`timerFunc`：用什么来实现“下一次事件循环”？

Vue 内部会优先选择「**微任务**」，不行再退而求其次到「**宏任务**」，大致优先级是：

1. `Promise.then` （微任务）
2. `MutationObserver` （微任务）
3. `setImmediate` （宏任务，IE 等环境）

4. `setTimeout(fn, 0)` （宏任务，兜底方案）

伪代码：

```
let timerFunc

if (typeof Promise !== 'undefined') {
  timerFunc = () => {
    Promise.resolve().then(flushCallbacks)
  }
} else if (typeof MutationObserver !== 'undefined') {
  const observer = new MutationObserver(flushCallbacks)
  const textNode = document.createTextNode('1')
  observer.observe(textNode, { characterData: true })
  timerFunc = () => {
    textNode.data = textNode.data === '1' ? '2' : '1'
  }
} else if (typeof setImmediate !== 'undefined') {
  timerFunc = () => {
    setImmediate(flushCallbacks)
  }
} else {
  timerFunc = () => {
    setTimeout(flushCallbacks, 0)
  }
}

```

Vue 2 vs Vue 3 的差异（简单了解）

- Vue 2：
  - 上面说的 watcher 队列 + `nextTick` 实现；
  - 使用 `Promise/MutationObserver/setTimeout` 等来实现异步队列。
- Vue 3：
  - 用了全新的 `scheduler` ，但**核心思想完全一样**：
    - 状态变更时，不立即重新渲染，而是把 `effect` 放入任务队列；
    - 再用微任务/宏任务异步 flush；
    - `nextTick` 依旧是把回调放到这个「异步 flush 之后」的队列里。

如何理解成一句话?

`nextTick` = 利用微任务/宏任务实现的「DOM 更新后的回调」机制。

- Vue 把 DOM 更新安排在一个异步 flush 里；
- `nextTick` 把你的函数排在这个 DOM 更新之后执行；
- 所以你能在 `nextTick` 回调中拿到**已经更新好的 DOM**。